



MongoDB Sessional Notebook:

Session 3 - MongoDB Operators

Date: July 14, 2025

Platform: MongoDB Compass Shell (`mongosh`)

This notebook covers Session 3 topics on MongoDB operators, based on the *Managing Large Databases Using MongoDB* textbook by Aptech Limited. Each section includes a definition, explanation, example, and classwork exercise, using raw MongoDB shell commands executable in the MongoDB Compass Shell. A sessional class test is provided at the end to assess understanding.

Instructions

- Run all commands in the **Shell** tab of MongoDB Compass (uses `mongosh`).
- Copy and paste each command or block into the Shell and press **Enter**.
- Use the **Collections** tab in Compass to visually confirm changes.
- Run `show dbs` or `show collections` to list databases or collections.
- For **Classwork**, attempt the exercise before checking the hint.
- For the **Sessional Class Test**, write commands yourself before referring to suggested answers.
- Clear the Shell output using the trash icon if cluttered.

Lecture Topics

1. Comparison Operators
 2. Logical Operators
 3. Element Operators
 4. Array Operators
 5. Projection Operators
 6. Field Update Operators
 7. Other Operators
-

1. Comparison Operators

Definition

Comparison operators (`$eq` , `$ne` , `$gt` , `$gte` , `$lt` , `$lte` , `$in` , `$nin`) compare values in queries to filter documents based on field values.

Explanation

These operators allow precise filtering of documents. For example, `$eq` matches exact values, `$gt` matches values greater than a specified number, and `$in` checks if a field value is in an array of values. They are used in `find()` or `update()` queries.

Example

```
use sample_analytics
db.accounts.find({ limit: { $gte: 9000 } })
```

Expected Output (based on textbook, page 65, Figure 3.2):

```
{ "_id": ObjectId("..."), "account_id": 123456, "limit": 10000, ... }
```

Classwork

Find all documents in the `accounts` collection where `limit` is less than 5000.

Hint:

```
db.accounts.find({ limit: { $lt: 5000 } })
```

2. Logical Operators

Definition

Logical operators (`$and` , `$or` , `$not` , `$nor`) combine multiple conditions in queries to filter documents based on logical relationships.

Explanation

Logical operators combine query conditions. `$and` requires all conditions to be true, `$or` requires at least one to be true, `$not` inverts a condition, and `$nor` requires all conditions to be false. They are useful for complex filtering.

Example

```
use sample_analytics
db.accounts.find({
  $and: [
    { limit: { $lt: 7000 } },
    { products: { $in: ["InvestmentStock"] } }
  ]
})
```

Expected Output (based on textbook, page 69, Figure 3.5):

```
{ "_id": ObjectId("..."), "account_id": 789012, "limit": 6000, "products": ["InvestmentStock", ...]
```

Classwork

Find documents in the `accounts` collection where `limit` is not greater than or equal to 5000.

Hint:

```
db.accounts.find({ limit: { $not: { $gte: 5000 } } })
```

3. Element Operators

Definition

Element operators (`$exists` , `$type`) check for the presence or data type of fields in documents.

Explanation

`$exists` checks if a field exists (`true`) or not (`false`) in a document. `$type` filters documents based on a field's BSON type (e.g., "string", "number"). These are useful for validating document structures.

Example

```
use sample_analytics
db.accounts.find({ limit: { $exists: true } })
```

Expected Output (based on textbook, page 74, Figure 3.9):

```
{ "_id": ObjectId("..."), "account_id": 123456, "limit": 10000, ... }
```

Classwork

Find documents in the `accounts` collection where the `products` field is of type "array".

Hint:

```
db.accounts.find({ products: { $type: "array" } })
```

4. Array Operators

Definition

Array operators (`$all`, `$elemMatch`, `$size`) filter documents based on array field content or properties.

Explanation

`$all` matches documents where an array contains all specified values. `$elemMatch` matches documents where an array element satisfies multiple conditions. `$size` matches arrays of a specific length. These operators are key for querying complex arrays.

Example

```
use sample_analytics
db.accounts.find({ products: { $all: ["food", "electronics"] } })
```

Expected Output (based on textbook, page 78, Figure 3.13):

```
{ "_id": ObjectId("..."), "account_id": 345678, "products": ["food", "electronics", ...], ... }
```

Classwork

Find documents in the `accounts` collection where the `products` array has exactly 3 elements.

Hint:

```
db.accounts.find({ products: { $size: 3 } })
```

5. Projection Operators

Definition

Projection operators (`$` , `$elemMatch` , `$slice`) control which fields or array elements are returned in query results.

Explanation

Projection operators limit the output of `find()` queries. The `$` operator returns the first matching array element, `$elemMatch` projects array elements matching multiple conditions, and `$slice` returns a subset of an array. Use with the projection parameter in `find()` .

Example

```
use sample_training
db.StudentMarks.find(
  { marks: { $gt: 80 } },
  { name: 1, marks: { $slice: 2 }, _id: 0 }
)
```

Expected Output (based on textbook, page 81):

```
{ "name": "Robert", "marks": [85, 90] }
```

Classwork

Find documents in the `StudentMarks` collection where `marks` is greater than 85, returning only the first matching mark using the `$` projection operator.

Hint:

```
db.StudentMarks.find({ marks: { $gt: 85 } }, { name: 1, "marks.$": 1, _id: 0 })
```

6. Field Update Operators

Definition

Field update operators (`$set` , `$unset` , `$inc` , `$mul` , `$rename` , `$min` , `$max` , `$currentDate` , `$addToSet`) modify document fields in update operations.

Explanation

These operators update specific fields. `$set` sets a field value, `$unset` removes a field, `$inc` increments a number, `$rename` changes a field name, and `$addToSet` adds unique values to an array. Used in `updateOne()` or `updateMany()` .

Example

```
use sample_training
db.StudentMarks.updateOne(
  { name: "David" },
  { $set: { credit: 0 } }
)
```

Expected Output (based on textbook, page 87, Figure 3.24):

```
{ acknowledged: true, matchedCount: 1, modifiedCount: 1 }
```

Classwork

Rename the `credit` field to `score` in documents in the `StudentMarks` collection where `name` is "David".

Hint:

```
db.StudentMarks.updateOne({ name: "David" }, { $rename: { credit: "score" } })
```

7. Other Operators

Definition

Other operators include `$comment`, `$expr`, and `$regex`, which add metadata, enable expression-based comparisons, or perform pattern matching.

Explanation

`$comment` adds notes to queries for debugging. `$expr` allows aggregation expressions in queries. `$regex` matches field values against regular expressions. These enhance query flexibility.

Example

```
use sample_training
db.StudentMarks.find(
  { name: { $regex: "^R", $options: "i" } },
  { $comment: "Find names starting with R" }
)
```

Expected Output (based on textbook, page 94, Figure 3.34):

```
{ "_id": ObjectId("..."), "name": "Robert", "marks": [85, 90], ... }
```

Classwork

Find documents in the `StudentMarks` collection where `marks` is greater than `credit` using `$expr`.

Hint:

```
db.StudentMarks.find({ $expr: { $gt: ["$marks", "$credit"] } })
```

Sessional Class Test

Instructions

Complete the following tasks in the MongoDB Compass Shell to test your understanding of Session 3. Run each command, verify the output, and note the results. Write commands yourself before checking suggested answers.

Tasks

1. Create a database called `inventory` and a collection called `products` .
2. Insert three documents into the `products` collection with fields for `name` (String), `price` (Number), `categories` (Array), and `stock` (Number). Example:

```
{ name: "Laptop", price: 1000, categories: ["electronics", "computers"], stock: 10 }
```

.
3. Query `products` to find documents where `price` is greater than 500, returning only `name` and `price` .
4. Use `$or` to find documents in `products` where `stock` is less than 5 or `categories` includes "electronics".
5. Check if the `stock` field exists in all documents of the `products` collection.
6. Update documents in `products` where `name` is "Laptop" to set `status` to "Available".
7. Add "accessories" to the `categories` array for documents where `price` is less than 200, ensuring no duplicates.

Suggested Commands (Try First, Then Check)

```
use inventory
db.createCollection("products")
db.products.insertMany([
  { name: "Laptop", price: 1000, categories: ["electronics", "computers"], stock: 10 },
  { name: "Mouse", price: 50, categories: ["electronics", "accessories"], stock: 20 },
  { name: "Book", price: 30, categories: ["stationery"], stock: 5 }
])
db.products.find({ price: { $gt: 500 } }, { name: 1, price: 1, _id: 0 }).pretty()
db.products.find({
  $or: [
    { stock: { $lt: 5 } },
    { categories: "electronics" }
  ]
}).pretty()
db.products.find({ stock: { $exists: true } }).pretty()
db.products.updateMany({ name: "Laptop" }, { $set: { status: "Available" } })
db.products.updateMany({ price: { $lt: 200 } }, { $addToSet: { categories: "accessories" } })
```

Expected Outputs (Approximate)

1. { ok: 1 }
2. { acknowledged: true, insertedIds: [ObjectId("..."), ObjectId("..."), ObjectId("...")] }
3. [{ "name": "Laptop", "price": 1000 }]
4. [{ "name": "Laptop", ... }, { "name": "Mouse", ... }]
5. [{ "name": "Laptop", ... }, { "name": "Mouse", ... }, { "name": "Book", ... }]
6. { acknowledged: true, matchedCount: 1, modifiedCount: 1 }
7. { acknowledged: true, matchedCount: 2, modifiedCount: 2 }

Additional Notes

- **Verify Changes:** Use `show dbs` to list databases or `show collections` to list collections. The **Collections** tab in Compass displays databases and collections visually.
- **Errors:** Commands may return `false` or an error if collections/documents don't exist.

Ensure MongoDB is running and you have permissions.

- **Textbook Reference:** Examples are adapted from pages 63–97, using `sample_analytics` and `sample_training` databases for consistency with textbook datasets.
- **Re-running:** Re-running insert commands may create duplicates; drop collections (`db.products.drop()`) to reset.
- **Operator Syntax:** Ensure correct operator syntax (e.g., `$gt` , not `gt`). Case sensitivity matters for `$regex` options.

Troubleshooting

- **Connection:** Verify MongoDB connection in Compass's **Connect** dialog.
- **No Output:** Ensure the correct database is active (`use <dbName>`) and documents exist.
- **Permissions:** For remote MongoDB, confirm your user has read/write permissions.
- **Syntax Errors:** Check for missing brackets, commas, or incorrect operator names.

End of Sessional Notebook